Source: docs\LOYALTY.md | Export: 11 November 2026

Loyalty Points

Implementation notes for the existing COD and Razorpay checkout integration. The November coupon/loyalty audit changes no production activation, balances, settings, rules or indexes. See COUPON_LOYALTY_AUDIT.md for audit scope and verification.

Sources and activation

- `siteSettings/loyalty`: `schemaVersion: 1`, `revision`, `config`, server audit metadata. Configuration is edited in **Store Settings → Loyalty Points**, using the existing atomic shipping/login-settings save. Missing configuration means disabled, with null commercial rates. No rates are seeded automatically.
- Config: `enabled`, `earnSpendPaise`, `earnPoints`, `pointValuePaise`, `minRedeemPoints`, `maxPercent`, `allowCoupons`. Money is whole paise. All active rates must be positive bounded safe integers; percent is 1-100. Disabling retains configured terms and all customer records.
- `loyaltyAccounts/{uid}`: private projection `schemaVersion`, signed `net`, nonnegative `reserved`, `pending`, `revision`, server `updatedAt`. Available = max(net, 0); debt = max(-net, 0). There is no balance editor or customer-write API.
- `loyaltyAccounts/{uid}/ledger/{orderId}_{event}`: immutable order-linked entries with `kind`, signed available change `points`, `reservedDelta`, `pendingDelta`, reason and server timestamp. Kinds are Reserved, Redeemed, Earned, Released, Restored and Reversed. The acceptance entry also records pending earnings. No independently writable balance is exposed.
- `orders/{id}.loyalty`: immutable commercial terms, config revision, redeemed points/paise, earning basis and expected points, per-line after-coupon basis and point discount. Mutable lifecycle fields track stage, actual earned/reversed/restored points, returned line bases and refunded shipping. Order history and operation receipts remain authoritative.

After an eventual application release, the owner must explicitly save approved valid rates to enable new loyalty orders. No automatic historical backfill, tiers, expiry, referrals, transfers or manual balance adjustments are provided. A separate explicit-ID repair is documented in CLIENT_CHANGES.md. Existing orders without a loyalty snapshot remain unchanged.

Calculation and confirmation

Let M be merchandise subtotal minus the existing coupon discount. Shipping stays **pre-coupon subtotal strictly greater than the saved threshold**, with equality still charged. Loyalty never alters that calculation.

- Redemption cap = floor(min(floor(M × maxPercent / 100), M − 1 paise) / pointValuePaise), limited by spendable availability. Below the minimum points threshold, the cap is zero. Requested whole points are clamped to the cap and minimum; reductions/removal are explained in the returned review. Coupons block points when compatibility is disabled.
- Savings = applied points × pointValuePaise. At least ₹0.01 merchandise remains payable, since zero-payable COD orders are not supported by the existing collection interface. Shipping is never discounted.
- Expected earnings = floor((M − point savings) × earnPoints / earnSpendPaise). The existing model does not separately identify tax, so no new tax calculation is introduced.
- Currency arithmetic and proportional allocation use BigInt. Whole points are limited to 10^12; unsupported calculations fail rather than overflowing.
- Quotes are read-only. Apply/Remove Points triggers a trusted quote; nothing is reserved until successful order acceptance. A changed balance, settings revision, coupon, price, stock or address changes the fingerprint. Submission returns a fresh review for confirmation rather than silently placing a differently priced order.
- Checkout storage holds only requested points and confirmation/recovery data, never authoritative balances, rates or savings. Customer UID comes from the verified token. Customer-specific components abort stale requests and reset on account changes.

Transaction lifecycle

Order acceptance reads settings, balance, stock and customer data in one Firestore transaction. It deducts requested available points into `reserved`, adds pending earnings, writes ledger entries, snapshots the order, deducts stock and writes the existing operation receipt atomically. Two tabs cannot spend one balance. Failed transactions leave no reservation. Same-operation recovery returns the accepted order and does not deduct twice.

Delivered settles an eligible saved snapshot: reserved points become redeemed and pending earnings become available, regardless of COD collection. Payment and delivery remain separate: delivery never marks an order collected, earlier collection never earns points, and later collection never duplicates the delivery award. The stable `{orderId}_earned` entry and transaction protect retries and concurrency. Current settings do not requalify a historical order.

Whole-order pre-completion cancellation/rejection releases reserved points and removes pending earnings. Resolving an entire return before payment collection also cancels the entitlement and releases points. Existing orders settle/release/refund under their saved terms even while the global program is disabled. Fulfilled cash refunds use the existing external-refund record, not a new money-transfer mechanism.

Returns and external refunds

For loyalty orders the existing **Record external manual refund** action now requires `loyaltyRefund: {lines: [{line, basisPaise}], shippingPaise}`. Values are **incremental**, not cumulative. A line identifies its original order position. Basis is returned merchandise value **after coupons and before points**. Empty allocations, duplicate line indexes, over-returns, over-shipping and unsafe amounts are rejected. Legacy orders retain their existing manual-refund flow.

Original after-coupon merchandise and point savings are allocated by cumulative proportional floors in original line order, so all line amounts sum exactly. For each line's cumulative returned basis R:

- Returned point-discount paise = floor(original line discount × R / original line eligible basis).
- Returned cash merchandise = R − returned point-discount paise.
- External cash amount must exactly equal the incremental returned cash merchandise plus incremental shipping refund. Points discounts cannot be refunded as cash. A genuine points-only allocation may have zero cash; an empty allocation cannot.
- Cumulative restored points = floor(original redeemed points × total returned point-discount paise / original point discount).
- Cumulative reversed earnings = floor(original earned points × cumulative returned cash merchandise / original earning basis).

Ledger changes are differences from saved cumulative targets, capped by the original order. Shipping-only refunds neither restore merchandise points nor reverse earnings. Full merchandise returns restore all redeemed points and reverse all earnings; sequential partial events cannot exceed either amount. Existing refund reference uniqueness, order revisions, receipts and confirmation/retry controls prevent duplicates.

If earnings were already spent, reversals remain in the ledger and can make `net` negative. Availability remains zero until new earnings or legitimate releases/restorations offset the debt. Other order snapshots are not rewritten.

The original request approval/resolution prerequisite is retained. The existing domain provides whole-order cancellations; no partial-cancellation endpoint is introduced. This adds line-value allocation to **financial refund recording**, not a partial stock return engine. Existing Receive and Restock still returns the entire order inventory. For a partial financial return, resolve the approved request without restocking, then record the allocated external refund. There is no automated courier synchronization, exchange replacement engine, online refund processor or new policy promise. For a collected order, a resolution note alone does not prove a cash refund or reverse earned points: the trusted allocated refund record does. For an unpaid delivered order, resolving its return reverses the delivery earnings and restores any redeemed points atomically, without claiming a cash refund. Stable unpaid-return ledger entries prevent repetition and preserve debt handling.

Screens and security

- Store Settings: rates, switches, draft example, Save/Cancel and revision conflicts.
- Checkout: available value, minimum/cap, Apply/Remove, warnings, savings and pending expectation.
- Customer Account and Admin Customer Details: availability/value, pending/reserved/debt and 20-entry paginated history with IST timestamps and owned order links. Customers list shows a compact available-points figure.
- Customer/Admin Order Details: snapshot savings, pending/earned/cancelled/reversed/restored state. The external-refund form includes line and shipping allocation controls.

The existing `/api/catalog` function is reused. `loyalty:account` and `loyalty:customerHistory` target the token UID and reject client UID fields. `loyalty:adminAccount`, `loyalty:adminHistory` and settings writes require both `admin: true` and enabled `adminAccess/{uid}`. Administrators still cannot edit balances directly.

The repository's existing default-deny rules cover `loyaltyAccounts`, its ledger and `siteSettings/loyalty`. Emulator tests reject direct reads/writes for signed-out users, customers and claimed administrators. **No rule changes or publication are required by this implementation under those existing rules.** The historical `loyalty` admin-read collection is not used for the new private projection. Hosted rule state has not been re-audited here.

Ledger history is a per-account collection query ordered by `createdAt DESC, __name__ DESC`; the document-name direction follows the existing single-field index. No new composite indexes are needed. Cursor documents are looked up only within the authorized account's ledger. Existing customer-directory/order indexes remain prerequisites of those unchanged modules.

Validation

`src/loyalty/model.test.ts` covers configuration bounds, disabled state, caps/minimums/coupons, floor rounding, exact line totals, cumulative partial/full refunds, shipping-only and points-only refunds, and debt arithmetic.

`scripts/loyalty-emulator.test.ts` uses loopback emulators and the real services/HTTP handler for atomic acceptance, collection eligibility, duplicate receipt recovery, competing orders, stale confirmations, cancellation/return release, snapshot preservation while disabled, allocated refunds, already-spent reversals, ledger/projection reconciliation, history pagination and ownership/dual authorization/direct-rule denial.

`scripts/loyalty-browser.test.py` uses loopback authentication, catalog API and Firestore for settings, Account retry, Apply/Remove, staged checkout, accepted order and refund UI, administrator/customer summaries, sign-out cleanup and widths 320-1920. Its fixture tool refuses non-loopback environments and creates only local orders through the trusted service.

These checks do not establish production activation or hosted acceptance. Any future release should first verify the existing production deny rules, then save owner-approved settings and review the feature through authorized sessions. No production test orders are needed for local validation.

Delivery earning change and historical review

See CLIENT_CHANGES.md for the explicit-ID, default-dry-run repair, exclusion criteria and local-only evidence. Already delivered pending orders are not credited by unrelated edits or a later collection event. Production historical repair requires a separate explicit instruction.